ENPM818J – Midterm Exam 1

Q1: Describe Real-Time scheduling algorithms in brief

A1:

Real-Time scheduling algorithms are used to determine when a task should run based on some priority of the task's parameters – execution time, period, and deadline. RT scheduling algorithms can prioritize different parameters of a task, and doing so leads to different scheduling behaviors, such as whether or not the schedule will be dynamic with respect to time, or static for all time. As such, there are two main types of RT scheduling methods: static and dynamic.

Static scheduling methods include:

- Rate Monotonic Scheduling (RMS)
    - prioritizes smaller (or shorter) task periods. Tasks with more frequent periods are executed over tasks with less frequent periods (as the name implies), always. A task with a more frequent period will always preempt a task that runs less frequently, even if the more infrequent task's deadline is approaching. As in – if Task 1 has a 1.5s deadline and a 1s execution time, and runs every 2 seconds, and task 2 has am execution time of 1s, a 2s deadline, and runs every 3 seconds, than at t=6, Task 1 will run before Task 2, even though Task 2 is right on the edge of it's deadline if it doesn't run at t=6 exactly
- Deadline Monotonic Scheduling (DMS) – prioritizes
    - prioritizes smaller (or shorter) task deadlines. Tasks with more frequent deadlines are executed over tasks with longer deadlines, always, and are not preempted based on how soon the deadline is approaching, but how short/long the deadline value of the task is. If a current task is nearing its deadline, but a task with a shorted deadline value needs to run as its period is coming up, then the current task will be preempted, EVEN though it's relative deadline is approaching
- Cyclic Executive (CE)
    - Follows a set update rate where pre-determined tasks are run within said loop rate. As in, if task 1 has a period of 5s, and task 2 has a period of 20 seconds, then the update loop rate (called the major cycle) will run every 20 seconds (for task 2), and within those 20 seconds, task 1 will run 4 times

Dynamic Scheduling methods include:

- Earliest Deadline First (EDF)
    - Prioritizes the most current, upcoming absolute deadline that a task has. As in, if a currently running tasks has a far away deadline, and a task that needs to be run has a much shorter deadline, it will preempt the task with the far away deadline to run the task with the shorter deadline. But, if the task that currently running's deadline is longer, but the task is closer to hitting the deadline, it will not be preempted. The time to the upcoming deadline is absolute and based on when the task starts (it's period). For example, if task 1 has a deadline of 4s, and task 2 has a deadline of 10 seconds, and task 2 has been running for 4s (with e2=8s) when task 1 needs to start,

then task 1 will preempt task 2 as it has a closer, absolute deadline (4s) than task 2's current relative deadline (6 seconds)

- Least Laxity First (LLF)
    - Prioritizes the tasks that's deadline is closest to being missed based on the upcoming deadline and remaining execution time, or chooses the task based on following expressions: (deadline time – current time) – remaining execution time. Different from EDF as it considers the remaining execution time to try and ensure no process will miss its deadline, while still dynamically scheduling them

Q2: Describe Process States and draw a process state diagram

A2:

Process states describe what point a process is within its lifecycle.

- A process starts at the "new" or "formed" state, which is the process being created and static memory being allocated to it, and identifying information being assigned like the process ID and the starting address (forms the process control block {PCB})
- The "ready" state is when the process has finished being formed and is ready to be executed by the scheduler
- The "running" state is the main state of the process, where code is being executed

A process can leave the running state and go to 3 other states:

- The "waiting" state, where a process is running but blocking as it's waiting for an input from something (user, I/O, file). After the block is passed, the process returns to the "ready" state
- The "ready" state again, where the process is interrupted and needs to be executed by the scheduler when it's ready
- The "terminated" state, where the process is closed, and the memory allocated to it is freed

State Diagram (see attached)

Q3: Explain Semaphores, briefly

A3:

Semaphores are integer values which are used to restrict/enable sections of threads to run based on the current status of other threads running at the same time. Using waiting and signaling functionality, a thread with a semaphore implemented must wait for the semaphore value to be updated to 1 (if it is not already 1) before running a function or section of code. If the value is 1, the thread gets access to the semaphore and sets it to 0 and runs the section of code. After running that section, the thread MUST update the semaphore value for others by signaling to the system that they have released the semaphore (and set it back to 1). This is how binary semaphores work, but there are also counting semaphores, which do the same thing except instead of one semaphore that can be acquired, multiple semaphores are available, allowing multiple different threads to run their section of code at the same time, up to the set number of semaphores. The waiting and signaling process is followed like in the binary semaphore. Since multiple threads may be waiting

for a semaphore to be freed up, they are typically placed in a queue – though I'm not positive if this is a FIFO queue of a FILO queue.

Q4: Verify the schedulability under EDF and construct EDF schedule of the following tasks set in the interval (0 to 20 units of time). Draw a timing diagram showing task execution for 20 units of time. Consider the following task set: $\tau 1 \equiv \{1, 5, 4\}$, $\tau 2 \equiv \{2, 8, 6\}$, and $\tau 3 \equiv \{1, 4, 3\}$. Here, the notation $\tau i \equiv \{ ei , pi , Di \}$ gives the execution time ei , period pi , and relative deadline Di of task τi

A4:

See attached excel sheet

Q5: Explain RM scheduling. Verify the schedulability under RM and construct the schedule of the task set Q4. Draw a timing diagram showing task execution for 20 units of time.

A5:

See answer to Q1, and see excel sheet for schedulability and timing diagram

Romeo Perlstein
ENPM818J - Real Time Operating Systems
College Park, MD 20742

HW2, Week 3

PROMPT: Q. Verify the schedulability under EDF and construct EDF schedule of the following tasks set in the interval (0 to 20 units of time). Draw a timing diagram showing task execution for 20 units of time. Consider the following task set: τ1 ≡ {1, 5, 4}, τ2 ≡ {2, 8, 6}, and τ3 ≡ {1, 4, 3}. Here, the notation τi ≡ { ei , pi , Di } gives the execution time, ei , period, pi , and relative deadline, Di , of task τi .

Task	ei	pi	Di	ui
τ1	1	5	4	0.2
τ2	2	8	6	0.25
τ3	1	4	3	0.25

Q4

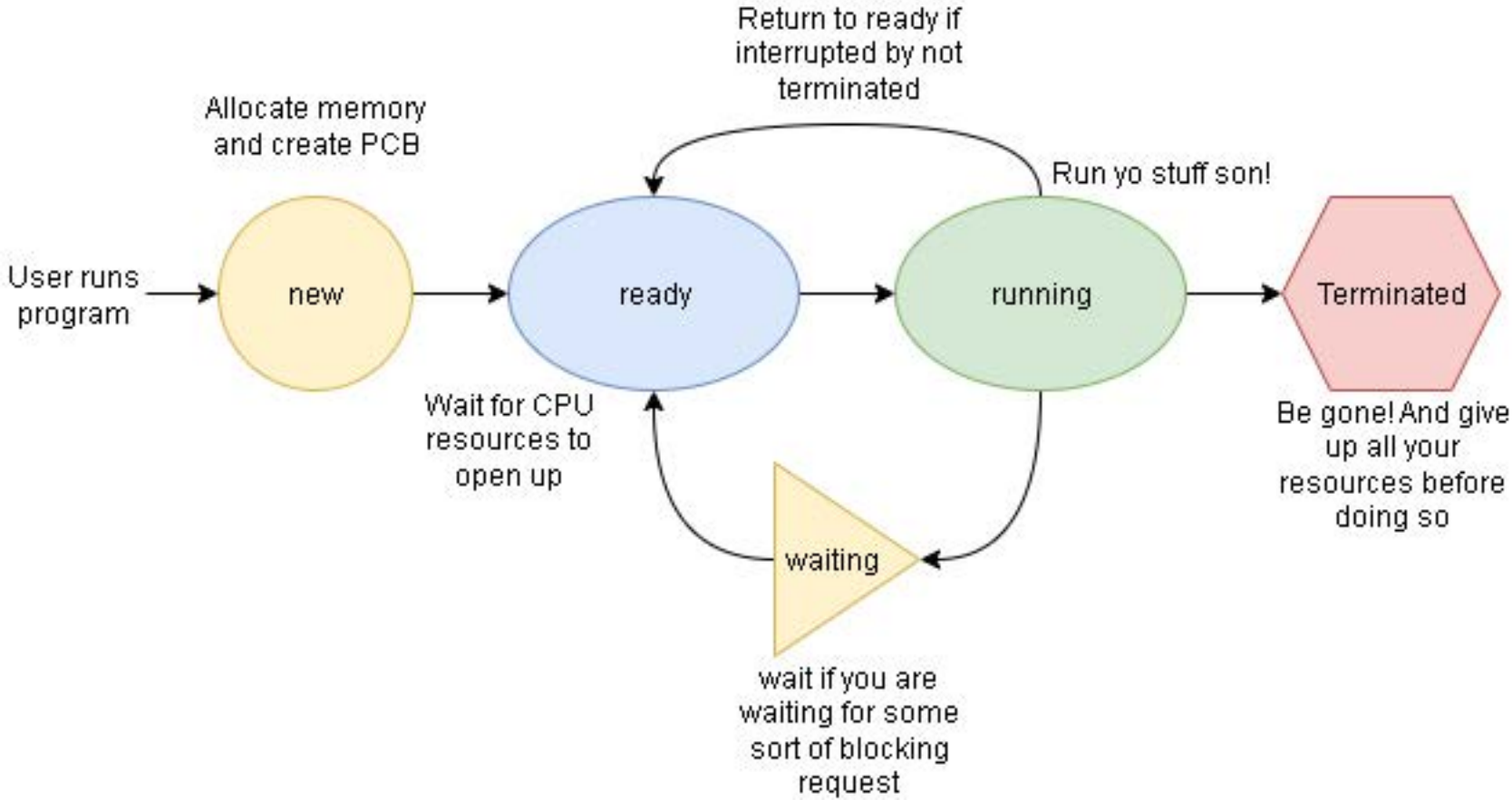
Time = 0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
Task 1 (τ {1, 5, 4}																											
Task 2 (τ {2, 8,6}																											
Task 3 (τ {1,4,3}																											
Timeline	τ3 starts, finishes	τ1 starts, finishes	τ2 starts	τ2 finishes	τ3 starts late	τ1 starts late		τ3 starts late	τ2 starts late	τ1 pre-empts τ2	τ2 starts again, finishes	τ3 starts	No tasks running		τ1 starts, finishes	τ3 starts, finishes	τ2 starts late	τ2 finishes	No tasks running	τ3 starts, finishes	τ1 starts late	No tasks running		τ3 starts, finishes	τ1 starts late	τ2 starts late	τ2 finishes

Schedulability	
Σ ui =	0.7
Tasks are schedulable	

Q5

Time = 0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
Task 1 (τ {1, 5, 4}																											
Task 2 (τ {2, 8,6}																											
Task 3 (τ {1,4,3}																											
Timeline	τ3 starts, finishes	τ1 starts, finishes	τ2 starts	τ3 starts, pre-empts τ2	τ3 starts, pre-empts τ2	τ2 starts again, finishes		τ3 starts, preempt τ2	τ2 starts late	τ1 starts pre-empts τ2	τ2 starts again, finishes	τ3 starts	No tasks running		τ1 starts, finishes	τ3 starts, pre-empts τ2	τ2 starts late	τ2 finishes	No tasks running	τ3 starts, finishes	τ1 starts late	No tasks running		τ3 starts, pre-empts τ2	τ1 starts pre-empts τ2	τ2 starts late	τ2 finishes

Schedulability		
Σ ui =	0.7	
n_val	0.7798	<------ 3(2^(1/3)-1)
ui < n_val!		
Task is schedulable!		


Return to ready if interrupted by not terminated
Allocate memory and create PCB
Run yo stuff son!
User runs program
new
ready
running
Terminated
Wait for CPU resources to open up
Be gone! And give up all your resources before doing so
waiting
wait if you are waiting for some sort of blocking request